

TỔNG HỢP MÃ NGUỒN

21 tệp | 1723 dòng mã nguồn | 10/09/2026

Toàn bộ tệp trong src, platformio.ini và partitions.csv.
Dòng dài được xuống dòng; số bên trái là số dòng gốc.

Mục lục

1. platformio.ini	2
2. partitions.csv	3
3. src/dht22_sensor.cpp	4
4. src/dht22_sensor.h	7
5. src/gateway_cloud.cpp	8
6. src/gateway_cloud.h	9
7. src/gateway_logic.cpp	10
8. src/gateway_logic.h	12
9. src/gateway_lora.cpp	13
10. src/gateway_lora.h	17
11. src/gateway_main.cpp	18
12. src/gateway_state.cpp	19
13. src/gateway_state.h	20
14. src/gateway_web.cpp	21
15. src/gateway_web.h	25
16. src/node_main.cpp	26
17. src/packet_protocol.cpp	32
18. src/packet_protocol.h	36
19. src/project_config.h	38
20. src/soil_moisture.cpp	39
21. src/soil_moisture.h	42

platformio.ini

```
1 [platformio]
2 default_envs = gateway
3
4 [env]
5 platform = espressif32
6 board = esp32doit-devkit-v1
7 framework = arduino
8 board_build.partitions = partitions.csv
9 monitor_speed = 115200
10 upload_speed = 921600
11 build_flags =
12     -D CORE_DEBUG_LEVEL=1
13 lib_deps =
14     sandeepmistry/LoRa
15
16 ; =====
17 ; NODE: sensors + LoRa telemetry + ACK/command + adaptive sleep
18 ; =====
19 [env:node]
20 build_src_filter =
21     -<*>
22     +<node_main.cpp>
23     +<dht22_sensor.cpp>
24     +<soil_moisture.cpp>
25     +<packet_protocol.cpp>
26
27 ; =====
28 ; GATEWAY: LoRa RX/TX + local dashboard/export + ThingsBoard HTTP
29 ; =====
30 [env:gateway]
31 build_src_filter =
32     -<*>
33     +<gateway_main.cpp>
34     +<gateway_state.cpp>
35     +<gateway_logic.cpp>
36     +<gateway_cloud.cpp>
37     +<gateway_lora.cpp>
38     +<gateway_web.cpp>
39     +<packet_protocol.cpp>
```

partitions.csv

1	#	Name,	Type,	SubType,	Offset,	Size,	Flags
2	nvs,		data,	nvs,	0x9000,	0x5000,	
3	app0,		app,	factory,	0x10000,	0x280000,	
4	spiffs,		data,	spiffs,	0x290000,	0x170000,	

src/dht22_sensor.cpp

```
1  #include "dht22_sensor.h"
2
3  #include <math.h>
4
5  void Dht22Sensor::begin(uint8_t pin, float tempOffsetC, float humidityOffsetRh)
6  {
7      _pin = pin;
8      _tempOffsetC = tempOffsetC;
9      _humidityOffsetRh = humidityOffsetRh;
10     pinMode(_pin, INPUT_PULLUP);
11     _started = true;
12 }
13
14 DhtReading Dht22Sensor::read()
15 {
16     if (!_started)
17     {
18         return fail();
19     }
20
21     uint8_t bytes[5] = {};
22     if (!readRaw40Bits(bytes))
23     {
24         return fail();
25     }
26
27     const uint8_t checksum = (bytes[0] + bytes[1] + bytes[2] + bytes[3]) & 0xFF;
28     if (checksum != bytes[4])
29     {
30         return fail();
31     }
32
33     const uint16_t rawHumidity = (uint16_t(bytes[0]) << 8) | bytes[1];
34     uint16_t rawTemperature = (uint16_t(bytes[2]) << 8) | bytes[3];
35     const bool negative = (rawTemperature & 0x8000) != 0;
36     rawTemperature &= 0x7FFF;
37
38     float humidityRh = rawHumidity / 10.0f + _humidityOffsetRh;
39     float temperatureC = rawTemperature / 10.0f + _tempOffsetC;
40     if (negative)
41     {
42         temperatureC = -temperatureC;
43     }
44
45     if (temperatureC < -10.0f || temperatureC > 50.0f || humidityRh < 0.0f || humidityRh > 100.0f)
46     {
47         return fail();
48     }
49
50     if (_hasPrevious &&
51         (fabsf(temperatureC - _previousTemperatureC) > 2.0f ||
52          fabsf(humidityRh - _previousHumidityRh) > 5.0f))
53     {
54         return fail();
55     }
56
57     pushSample(temperatureC, humidityRh);
58     _previousTemperatureC = temperatureC;
59     _previousHumidityRh = humidityRh;
60     _hasPrevious = true;
61
62     DhtReading reading;
```

src/dht22_sensor.cpp

```
63     reading.temperatureC = roundf(average(_tempBuffer) * 10.0f) / 10.0f;
64     reading.humidityRh = roundf(average(_humidityBuffer) * 10.0f) / 10.0f;
65     reading.status = SensorStatus::Ok;
66     reading.errorFlag = 0;
67     return reading;
68 }
69
70 bool Dht22Sensor::readRaw40Bits(uint8_t bytes[5])
71 {
72     pinMode(_pin, OUTPUT_OPEN_DRAIN);
73     digitalWrite(_pin, LOW);
74     delay(20);
75     digitalWrite(_pin, HIGH);
76     delayMicroseconds(40);
77     pinMode(_pin, INPUT_PULLUP);
78
79     uint32_t duration = 0;
80     if (!expectPulse(LOW, 100, duration) || !expectPulse(HIGH, 100, duration))
81     {
82         return false;
83     }
84
85     for (uint8_t bitIndex = 0; bitIndex < 40; bitIndex++)
86     {
87         if (!expectPulse(LOW, 80, duration) || !expectPulse(HIGH, 100, duration))
88         {
89             return false;
90         }
91
92         if (duration > 50)
93         {
94             bytes[bitIndex / 8] |= (1 << (7 - (bitIndex % 8)));
95         }
96     }
97
98     return true;
99 }
100
101 bool Dht22Sensor::expectPulse(uint8_t level, uint32_t timeoutUs, uint32_t &durationUs) const
102 {
103     const uint32_t start = micros();
104     while (digitalRead(_pin) != level)
105     {
106         if (micros() - start > timeoutUs)
107         {
108             return false;
109         }
110     }
111
112     const uint32_t pulseStart = micros();
113     while (digitalRead(_pin) == level)
114     {
115         if (micros() - pulseStart > timeoutUs)
116         {
117             return false;
118         }
119     }
120
121     durationUs = micros() - pulseStart;
122     return true;
123 }
124
```

src/dht22_sensor.cpp

```
125 DhtReading Dht22Sensor::fail() const
126 {
127     DhtReading reading;
128     reading.status = SensorStatus::Error;
129     reading.errorFlag = 1;
130     return reading;
131 }
132
133 void Dht22Sensor::pushSample(float temperatureC, float humidityRh)
134 {
135     _tempBuffer[_bufferIndex] = temperatureC;
136     _humidityBuffer[_bufferIndex] = humidityRh;
137     _bufferIndex = (_bufferIndex + 1) % kAverageSize;
138     if (_bufferCount < kAverageSize)
139     {
140         _bufferCount++;
141     }
142 }
143
144 float Dht22Sensor::average(const float *values) const
145 {
146     float sum = 0.0f;
147     for (size_t i = 0; i < _bufferCount; i++)
148     {
149         sum += values[i];
150     }
151     return _bufferCount == 0 ? NAN : sum / _bufferCount;
152 }
```

src/dht22_sensor.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  enum class SensorStatus : uint8_t
6  {
7      Ok = 0,
8      Error = 1
9  };
10
11 struct DhtReading
12 {
13     float temperatureC = NAN;
14     float humidityRh = NAN;
15     SensorStatus status = SensorStatus::Error;
16     uint8_t errorFlag = 1;
17 };
18
19 class Dht22Sensor
20 {
21 public:
22     void begin(uint8_t pin, float tempOffsetC, float humidityOffsetRh);
23     DhtReading read();
24
25 private:
26     static constexpr size_t kAverageSize = 5;
27
28     uint8_t _pin = 0;
29     float _tempOffsetC = 0.0f;
30     float _humidityOffsetRh = 0.0f;
31     bool _started = false;
32     bool _hasPrevious = false;
33     float _previousTemperatureC = 0.0f;
34     float _previousHumidityRh = 0.0f;
35     float _tempBuffer[kAverageSize] = {};
36     float _humidityBuffer[kAverageSize] = {};
37     size_t _bufferCount = 0;
38     size_t _bufferIndex = 0;
39
40     bool readRaw40Bits(uint8_t bytes[5]);
41     bool expectPulse(uint8_t level, uint32_t timeoutUs, uint32_t &durationUs) const;
42     DhtReading fail() const;
43     void pushSample(float temperatureC, float humidityRh);
44     float average(const float *values) const;
45 };
```

src/gateway_cloud.cpp

```
1  #include "gateway_cloud.h"
2
3  #include <Arduino.h>
4  #include <HTTPClient.h>
5  #include <WiFi.h>
6
7  #include "gateway_logic.h"
8  #include "project_config.h"
9
10 void uploadThingsBoard(const TelemetryPacket &packet, int rssi)
11 {
12     if (!Config::EnableCloudUpload || WiFi.status() != WL_CONNECTED ||
13         String(Config::ThingsBoardToken).length() == 0)
14     {
15         return;
16     }
17
18     HTTPClient http;
19     http.setTimeout(1500);
20     const String url = String("http://") + Config::ThingsBoardHost + ":" +
21         String(Config::ThingsBoardHttpPort) + "/api/v1/" +
22         Config::ThingsBoardToken + "/telemetry";
23     http.begin(url);
24     http.addHeader("Content-Type", "application/json");
25     const String body = String("{\"Node_ID\":") + packet.nodeId +
26         "\",\"T_air\":") + String(packet.temperatureC, 1) +
27         "\",\"H_air\":") + String(packet.humidityRh, 1) +
28         "\",\"H_soil\":") + String(packet.soilMoistureVol, 0) +
29         "\",\"RSSI\":") + String(rssi) +
30         "\",\"Error_Flag\":") + String(packet.errorFlag) +
31         "\",\"Alert\":\"\") + alertLevel(packet) + \"\"\" +
32         "\",\"PumpTime\":") + String(computePumpTime(packet)) +
33         "\",\"Sleep_Min\":") + String(packet.configSleepMinutes) +
34         "\",\"Soil_Threshold\":") + String(packet.configSoilThresholdVol) +
35         "\",\"Filter_Mode\":") + String(packet.configFilterMode) +
36         "\",\"Config_Pump_Time\":") + String(packet.configPumpSeconds) +
37         "\",\"Control_Mode\":") + String(packet.configControlMode) +
38         "\",\"Duty_Cycle\":") + String(packet.configDutyCycleMode) + "\"}";
39     const int status = http.POST(body);
40     Serial.printf("ThingsBoard HTTP status: %d\n", status);
41     http.end();
42 }
```


src/gateway_cloud.h

```
1  #pragma once
2
3  #include "packet_protocol.h"
4
5  void uploadThingsBoard(const TelemetryPacket &packet, int rssi);
```

src/gateway_logic.cpp

```
1  #include "gateway_logic.h"
2
3  #include "gateway_state.h"
4
5  int parseCommandParameter(CommandType command, String value)
6  {
7      value.trim();
8      value.toUpperCase();
9      if (command == CommandType::SetFilterMode)
10     {
11         return value == "MEDIAN" ? 1 : 0;
12     }
13     if (command == CommandType::SetControlMode)
14     {
15         return value == "AUTO" ? 1 : 0;
16     }
17     if (command == CommandType::SetDutyCycle)
18     {
19         return value == "ADAPTIVE" ? 1 : 0;
20     }
21     return value.toInt();
22 }
23
24 int computePumpTime(const TelemetryPacket &packet)
25 {
26     if (packet.errorFlag != 0)
27     {
28         return 0;
29     }
30
31     int pumpTime = 0;
32     if (packet.soilMoistureVol > 42.0f)
33     {
34         pumpTime = 0;
35     }
36     else if (packet.soilMoistureVol > 33.0f)
37     {
38         pumpTime = 0;
39     }
40     else if (packet.soilMoistureVol > 24.0f)
41     {
42         pumpTime = 3;
43     }
44     else if (packet.soilMoistureVol > 15.0f)
45     {
46         pumpTime = 6;
47     }
48     else
49     {
50         pumpTime = 10;
51     }
52
53     if (packet.temperatureC > 32.0f)
54     {
55         pumpTime += 2;
56     }
57     if (packet.humidityRh < 50.0f)
58     {
59         pumpTime += 3;
60     }
61     if (packet.humidityRh > 80.0f)
62     {
```

src/gateway_logic.cpp

```
63     pumpTime -= 2;
64 }
65 return constrain(pumpTime, 0, 15);
66 }
67
68 String alertLevel(const TelemetryPacket &packet)
69 {
70     if (packet.errorFlag != 0)
71     {
72         return "SENSOR_ERROR";
73     }
74     if (packet.soilMoistureVol > 42.0f)
75     {
76         return "OVER_MOISTURE";
77     }
78     if (packet.soilMoistureVol > 33.0f)
79     {
80         return "NORMAL";
81     }
82     if (packet.soilMoistureVol > 24.0f)
83     {
84         return "LIGHT_DRY";
85     }
86     if (packet.soilMoistureVol > 15.0f)
87     {
88         return "NEED_WATERING";
89     }
90     return "URGENT_WATERING";
91 }
92
93 void storeTelemetry(const TelemetryPacket &packet, int rssi, float snr)
94 {
95     history[historyWriteIndex].packet = packet;
96     history[historyWriteIndex].rssi = rssi;
97     history[historyWriteIndex].snr = snr;
98     history[historyWriteIndex].receivedMs = millis();
99     historyWriteIndex = (historyWriteIndex + 1) % kGatewayHistorySize;
100     if (historyCount < kGatewayHistorySize)
101     {
102         historyCount++;
103     }
104 }
105
106 void queueGatewayCommand(uint8_t nodeId, CommandType command, int parameter)
107 {
108     portENTER_CRITICAL(&pendingCommandMux);
109     pendingCommands[nodeId].nodeId = nodeId;
110     pendingCommands[nodeId].command = command;
111     pendingCommands[nodeId].parameter = parameter;
112     pendingCommands[nodeId].ok = true;
113     pendingCommands[nodeId].status = "QUEUED";
114     pendingCommandRepeats[nodeId] = 3;
115     portEXIT_CRITICAL(&pendingCommandMux);
116 }
```

src/gateway_logic.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  #include "packet_protocol.h"
6
7  int parseCommandParameter(CommandType command, String value);
8  int computePumpTime(const TelemetryPacket &packet);
9  String alertLevel(const TelemetryPacket &packet);
10 void storeTelemetry(const TelemetryPacket &packet, int rssi, float snr);
11 void queueGatewayCommand(uint8_t nodeId, CommandType command, int parameter);
```

src/gateway_lora.cpp

```
1  #include "gateway_lora.h"
2
3  #include <Arduino.h>
4  #include <LoRa.h>
5  #include <SPI.h>
6
7  #include "gateway_cloud.h"
8  #include "gateway_logic.h"
9  #include "gateway_state.h"
10 #include "packet_protocol.h"
11 #include "project_config.h"
12
13 bool initLoRa()
14 {
15     LoRa.end();
16     SPI.end();
17     delay(50);
18
19     pinMode(Config::LoraSs, OUTPUT);
20     digitalWrite(Config::LoraSs, HIGH);
21     pinMode(Config::LoraRst, OUTPUT);
22     digitalWrite(Config::LoraRst, LOW);
23     delay(50);
24     digitalWrite(Config::LoraRst, HIGH);
25     delay(200);
26
27     SPI.begin(Config::LoraSck, Config::LoraMiso, Config::LoraMosi, Config::LoraSs);
28     LoRa.setPins(Config::LoraSs, Config::LoraRst, Config::LoraDio0);
29     if (!LoRa.begin(Config::LoraFrequency))
30     {
31         LoRa.end();
32         SPI.end();
33         return false;
34     }
35
36     LoRa.setSpreadingFactor(Config::LoraSpreadingFactor);
37     LoRa.setSignalBandwidth(Config::LoraSignalBandwidth);
38     LoRa.setCodingRate4(Config::LoraCodingRate);
39     LoRa.setTxPower(Config::LoraTxPowerDbm);
40     LoRa.enableCrc();
41     LoRa.receive();
42     return true;
43 }
44
45 static void sendAckFor(const TelemetryPacket &packet)
46 {
47     if (!loraReady)
48     {
49         return;
50     }
51
52     AckPacket ack;
53     ack.nodeId = packet.nodeId;
54     ack.packetId = packet.packetId;
55     ack.ok = packet.errorFlag == 0;
56     ack.status = ack.ok ? "OK" : "SENSOR_ERROR";
57
58     CommandType queuedCommand = CommandType::None;
59     int queuedParameter = 0;
60     portENTER_CRITICAL(&pendingCommandMux);
61     queuedCommand = pendingCommands[packet.nodeId].command;
62     queuedParameter = pendingCommands[packet.nodeId].parameter;
```

src/gateway_lora.cpp

```
63     if (queuedCommand != CommandType::None)
64     {
65         if (pendingCommandRepeats[packet.nodeId] > 0)
66         {
67             pendingCommandRepeats[packet.nodeId]--;
68         }
69         if (pendingCommandRepeats[packet.nodeId] == 0)
70         {
71             pendingCommands[packet.nodeId].command = CommandType::None;
72             pendingCommands[packet.nodeId].parameter = 0;
73             pendingCommands[packet.nodeId].status = "SENT";
74         }
75     }
76     portEXIT_CRITICAL(&pendingCommandMux);
77
78     if (queuedCommand != CommandType::None)
79     {
80         ack.command = queuedCommand;
81         ack.parameter = queuedParameter;
82         ack.status = "COMMAND";
83     }
84     else if (Config::EnablePumpHardware)
85     {
86         const int pumpTime = computePumpTime(packet);
87         if (pumpTime > 0)
88         {
89             ack.command = CommandType::StartPump;
90             ack.parameter = pumpTime;
91         }
92     }
93
94     const String payload = encodeAck(ack);
95     LoRa.beginPacket();
96     LoRa.print(payload);
97     LoRa.endPacket();
98     LoRa.receive();
99     delay(100);
100    Serial.print("ACK TX: ");
101    Serial.println(payload);
102
103 }
104
105 void processLoRa()
106 {
107     if (!loraReady)
108     {
109         return;
110     }
111
112     const int packetSize = LoRa.parsePacket();
113     if (packetSize <= 0)
114     {
115         return;
116     }
117
118     String line;
119     while (LoRa.available())
120     {
121         line += char(LoRa.read());
122     }
123
124     TelemetryPacket packet;
```

src/gateway_lora.cpp

```
125     if (!decodeTelemetry(line, packet))
126     {
127         Serial.print("Invalid RF packet: ");
128         Serial.println(line);
129         return;
130     }
131
132     if (lastPacketIdByNode[packet.nodeId] == packet.packetId)
133     {
134         Serial.printf("Duplicate packet ignored: node=%u pid=%lu\n", packet.nodeId, packet.packetId)
135     ;
136         sendAckFor(packet);
137         return;
138     }
139     lastPacketIdByNode[packet.nodeId] = packet.packetId;
140
141     const int rssi = LoRa.packetRssi();
142     const float snr = LoRa.packetSnr();
143     storeTelemetry(packet, rssi, snr);
144
145     Serial.printf("DATA RX node=%u pid=%lu T=%.1f H=%.1f Soil=%.0f ADC=%d RSSI=%d SNR=%.1f Alert=%s
146 PumpTime=%d\n",
147                 packet.nodeId, packet.packetId, packet.temperatureC, packet.humidityRh,
148                 packet.soilMoistureVol, packet.adcFiltered, rssi, snr,
149                 alertLevel(packet).c_str(), computePumpTime(packet));
150
151     uploadThingsBoard(packet, rssi);
152     sendAckFor(packet);
153 }
154
155 void processSerialCommand()
156 {
157     if (!Serial.available())
158     {
159         return;
160     }
161
162     String line = Serial.readStringUntil('\n');
163     line.trim();
164     line.toUpperCase();
165
166     const int firstSpace = line.indexOf(' ');
167     const int secondSpace = line.indexOf(' ', firstSpace + 1);
168     if (firstSpace < 0)
169     {
170         Serial.println("Serial command format: NODE_ID CMD PARAM");
171         return;
172     }
173
174     const uint8_t nodeId = uint8_t(line.substring(0, firstSpace).toInt());
175     const String cmdText = secondSpace < 0 ? line.substring(firstSpace + 1)
176 : line.substring(firstSpace + 1, secondSpace);
177     const int parameter = secondSpace < 0 ? 0 : line.substring(secondSpace + 1).toInt();
178     const CommandType command = commandFromText(cmdText);
179     if (nodeId == 0 || command == CommandType::None)
180     {
181         Serial.println("Invalid serial command. Example: 1 START_PUMP 5");
182         return;
183     }
184
185     queueGatewayCommand(nodeId, command, parameter);
186     Serial.printf("Queued command for node %u: %s %d\n", nodeId, cmdText.c_str(), parameter);
```

src/gateway_lora.cpp

```
185 }
```


src/gateway_lora.h

```
1  #pragma once
2
3  bool initLoRa();
4  void processLoRa();
5  void processSerialCommand();
```

src/gateway_main.cpp

```
1  #include <Arduino.h>
2  #include <WiFi.h>
3
4  #include "gateway_lora.h"
5  #include "gateway_state.h"
6  #include "gateway_web.h"
7  #include "project_config.h"
8
9  void setup()
10 {
11     Serial.begin(Config::SerialBaud);
12     delay(1000);
13     Serial.println("==== EE4552 GATEWAY FIRMWARE =====");
14     Serial.print("Firmware: ");
15     Serial.println(Config::GatewayFirmwareVersion);
16
17     loraReady = initLoRa();
18     if (!loraReady)
19     {
20         Serial.println("LoRa init failed; WiFi/web will continue");
21     }
22     else
23     {
24         Serial.println("LoRa init OK");
25     }
26
27     connectWiFi();
28     setupWebServer();
29     startWebServerTask();
30     Serial.println("Gateway ready");
31 }
32
33 void loop()
34 {
35     if (Config::EnableGatewayHeartbeatLog && millis() - lastHeartbeatMs >= 2000)
36     {
37         lastHeartbeatMs = millis();
38         Serial.printf("Loop alive wifi=%d ip=%s\n", int(WiFi.status()), WiFi.localIP().toString().c_
39 str());
40     }
41
42     processLoRa();
43     processSerialCommand();
44     delay(1);
45 }
```

src/gateway_state.cpp

```
1  #include "gateway_state.h"
2
3  StoredTelemetry history[kGatewayHistorySize];
4  size_t historyCount = 0;
5  size_t historyWriteIndex = 0;
6  uint32_t lastPacketIdByNode[256] = {};
7  AckPacket pendingCommands[256];
8  uint8_t pendingCommandRepeats[256] = {};
9  portMUX_TYPE pendingCommandMux = portMUX_INITIALIZER_UNLOCKED;
10 uint32_t lastHeartbeatMs = 0;
11 uint32_t lastLoraRetryMs = 0;
12 bool loraReady = false;
```

src/gateway_state.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  #include "packet_protocol.h"
6
7  struct StoredTelemetry
8  {
9      TelemetryPacket packet;
10     int rssi = 0;
11     float snr = 0.0f;
12     uint32_t receivedMs = 0;
13 };
14
15 constexpr size_t kGatewayHistorySize = 64;
16
17 extern StoredTelemetry history[kGatewayHistorySize];
18 extern size_t historyCount;
19 extern size_t historyWriteIndex;
20 extern uint32_t lastPacketIdByNode[256];
21 extern AckPacket pendingCommands[256];
22 extern uint8_t pendingCommandRepeats[256];
23 extern portMUX_TYPE pendingCommandMux;
24 extern uint32_t lastHeartbeatMs;
25 extern uint32_t lastLoraRetryMs;
26 extern bool loraReady;
```

src/gateway_web.cpp

```
1  #include "gateway_web.h"
2
3  #include <Arduino.h>
4  #include <WebServer.h>
5  #include <WiFi.h>
6
7  #include "gateway_logic.h"
8  #include "gateway_state.h"
9  #include "packet_protocol.h"
10 #include "project_config.h"
11
12 static WebServer server(80);
13 static WiFiServer debugServer(8080);
14 static TaskHandle_t webTaskHandle = nullptr;
15
16 static void handleDebugServer();
17
18 void connectWiFi()
19 {
20     if (String(Config::WifiSsid) == "TEN_WIFI_CUA_BAN")
21     {
22         Serial.println("WiFi disabled: set WifiSsid/WifiPassword in project_config.h");
23         return;
24     }
25
26     WiFi.mode(WIFI_STA);
27     WiFi.setSleep(false);
28     WiFi.begin(Config::WifiSsid, Config::WifiPassword);
29     Serial.print("Connecting WiFi");
30     const uint32_t start = millis();
31     while (WiFi.status() != WL_CONNECTED && millis() - start < 15000)
32     {
33         Serial.print(".");
34         delay(500);
35     }
36     Serial.println();
37     if (WiFi.status() == WL_CONNECTED)
38     {
39         Serial.print("Gateway IP: ");
40         Serial.println(WiFi.localIP());
41     }
42 }
43
44 static String csvHistory()
45 {
46     String csv = "time_ms,node_id,packet_id,t_air,h_air,h_soil,adc,soil_status,error_flag,rssi,snr,s
47     leep_min,soil_threshold,filter_mode,pump_time,control_mode,duty_cycle\n";
48     for (size_t i = 0; i < historyCount; i++)
49     {
50         const size_t idx = (historyWriteIndex + kGatewayHistorySize - historyCount + i) % kGatewayHi
51         storySize;
52         const StoredTelemetry &item = history[idx];
53         csv += String(item.receivedMs) + "," + item.packet.nodeId + "," + item.packet.packetId + ","
54         ;
55         csv += String(item.packet.temperatureC, 1) + "," + String(item.packet.humidityRh, 1) + ",";
56         csv += String(item.packet.soilMoistureVol, 1) + "," + String(item.packet.adcFiltered) + ","
57         + item.packet.soilStatus + ",";
58         csv += String(item.packet.errorFlag) + "," + String(item.rssi) + "," + String(item.snr, 1) +
59         ",";
60         csv += String(item.packet.configSleepMinutes) + "," + String(item.packet.configSoilThreshold
61         Vol) + ",";
62         csv += String(item.packet.configFilterMode) + "," + String(item.packet.configPumpSeconds) +
```

```

    ",";
57     csv += String(item.packet.configControlMode) + "," + String(item.packet.configDutyCycleMode)
    + "\n";
58     }
59     return csv;
60 }
61
62 static void handleRoot()
63 {
64     if (Config::EnableGatewayHttpDebug)
65     {
66         Serial.println("HTTP GET /");
67     }
68     String html = "<!doctype html><html><head><meta charset='utf-8'><meta name='viewport' content='w
    idth=device-width,initial-scale=1'>";
69     html += "<title>EE4552 Gateway</title><style>body{font-family:Arial;margin:24px}table{border-col
    lapse:collapse;width:100%}td,th{border:1px solid #ccc;padding:6px;text-align:left}</style></head><bo
    dy>";
70     html += "<h2>EE4552 Gateway Dashboard</h2><p><a href='/export.csv'>Export CSV</a></p>";
71     html += "<p>Firmware: " + String(Config::GatewayFirmwareVersion) + "</p>";
72     html += "<p>Queue command: /command?node=1&cmd=SET_THRESHOLD&param=20 or SET_FILTER_MODE
    , SET_PUMP_TIME, SET_CONTROL_MODE</p>";
73     html += "<table><tr><th>ms</th><th>Node</th><th>PID</th><th>T</th><th>H_air</th><th>H_soil</th><
    th>Status</th><th>RSSI</th><th>Sleep</th><th>Threshold</th><th>Pump</th><th>Control</th><th>Duty</th>
    </tr>";
74     for (size_t i = 0; i < historyCount; i++)
75     {
76         const size_t idx = (historyWriteIndex + kGatewayHistorySize - historyCount + i) % kGatewayHi
    storySize;
77         const StoredTelemetry &item = history[idx];
78         html += "<tr><td>" + String(item.receivedMs) + "</td><td>" + String(item.packet.nodeId) + "<
    /td><td>" + String(item.packet.packetId) + "</td>";
79         html += "<td>" + String(item.packet.temperatureC, 1) + "</td><td>" + String(item.packet.humi
    dityRh, 1) + "</td><td>" + String(item.packet.soilMoistureVol, 1) + "</td>";
80         html += "<td>" + alertLevel(item.packet) + "</td><td>" + String(item.rssi) + "</td>";
81         html += "<td>" + String(item.packet.configSleepMinutes) + "</td><td>" + String(item.packet.c
    onfigSoilThresholdVol) + "</td>";
82         html += "<td>" + String(item.packet.configPumpSeconds) + "</td><td>" + String(item.packet.co
    nfigControlMode) + "</td>";
83         html += "<td>" + String(item.packet.configDutyCycleMode) + "</td></tr>";
84     }
85     html += "</table></body></html>";
86     server.send(200, "text/html", html);
87 }
88
89 static void handleExport()
90 {
91     if (Config::EnableGatewayHttpDebug)
92     {
93         Serial.println("HTTP GET /export.csv");
94     }
95     server.setHeader("Content-Disposition", "attachment; filename=ee4552_history.csv");
96     server.send(200, "text/csv", csvHistory());
97 }
98
99 static void handleCommand()
100 {
101     if (Config::EnableGatewayHttpDebug)
102     {
103         Serial.println("HTTP GET /command");
104     }
105     const uint8_t nodeId = uint8_t(server.arg("node").toInt());

```

src/gateway_web.cpp

```
106     const CommandType command = commandFromText(server.arg("cmd"));
107     int parameter = parseCommandParameter(command, server.arg("param"));
108
109     if (nodeId == 0 || command == CommandType::None)
110     {
111         server.send(400, "text/plain", "invalid node/cmd");
112         return;
113     }
114
115     queueGatewayCommand(nodeId, command, parameter);
116
117     Serial.printf("Queued HTTP command node=%u cmd=%s param=%d\n",
118                 nodeId, commandToText(command).c_str(), parameter);
119
120     server.send(200, "text/plain", "queued");
121 }
122
123 static void handlePing()
124 {
125     if (Config::EnableGatewayHttpDebug)
126     {
127         Serial.println("HTTP GET /ping");
128     }
129     server.send(200, "text/plain", "pong");
130 }
131
132 static void handleNotFound()
133 {
134     if (Config::EnableGatewayHttpDebug)
135     {
136         Serial.print("HTTP 404 ");
137         Serial.println(server.uri());
138     }
139     server.send(404, "text/plain", "not found");
140 }
141
142 void setupWebServer()
143 {
144     server.on("/", handleRoot);
145     server.on("/ping", handlePing);
146     server.on("/export.csv", handleExport);
147     server.on("/command", handleCommand);
148     server.onNotFound(handleNotFound);
149     server.begin();
150     Serial.println("HTTP server listening on port 80");
151     debugServer.begin();
152     Serial.println("Debug HTTP server listening on port 8080");
153 }
154
155 static void webServerTask(void *)
156 {
157     while (true)
158     {
159         handleDebugServer();
160         server.handleClient();
161         delay(2);
162     }
163 }
164
165 void startWebServerTask()
166 {
167     if (webTaskHandle != nullptr)
```

src/gateway_web.cpp

```
168     {
169         return;
170     }
171
172     xTaskCreatePinnedToCore(
173         webServerTask,
174         "web_server",
175         8192,
176         nullptr,
177         1,
178         &webTaskHandle,
179         0);
180 }
181
182 static void handleDebugServer()
183 {
184     WiFiClient client = debugServer.available();
185     if (!client)
186     {
187         return;
188     }
189
190     const uint32_t start = millis();
191     while (client.connected() && !client.available() && millis() - start < 1000)
192     {
193         delay(1);
194     }
195
196     String requestLine;
197     if (client.available())
198     {
199         requestLine = client.readStringUntil('\n');
200         requestLine.trim();
201     }
202
203     if (Config::EnableGatewayHttpDebug)
204     {
205         Serial.print("DEBUG HTTP request: ");
206         Serial.println(requestLine.length() > 0 ? requestLine : "(empty)");
207     }
208
209     client.print("HTTP/1.1 200 OK\r\n");
210     client.print("Content-Type: text/plain\r\n");
211     client.print("Connection: close\r\n");
212     client.print("Content-Length: 4\r\n\r\n");
213     client.print("pong");
214     client.stop();
215 }
```


src/gateway_web.h

```
1  #pragma once
2
3  void connectWiFi();
4  void setupWebServer();
5  void startWebServerTask();
```

src/node_main.cpp

```
1  #include <Arduino.h>
2  #include <LoRa.h>
3  #include <Preferences.h>
4  #include <SPI.h>
5
6  #include "dht22_sensor.h"
7  #include "packet_protocol.h"
8  #include "project_config.h"
9  #include "soil_moisture.h"
10
11 RTC_DATA_ATTR uint32_t rtcPacketId = 0;
12 RTC_DATA_ATTR uint32_t rtcSleepMinutes = Config::DefaultSleepMinutes;
13
14 static Dht22Sensor dht22;
15 static SoilMoistureSensor soilSensor;
16 static Preferences preferences;
17
18 struct NodeRuntimeConfig
19 {
20     int soilThresholdVol = 20;
21     uint32_t sleepMinutes = Config::DefaultSleepMinutes;
22     int filterMode = 0; // 0=AVERAGE, 1=MEDIAN.
23     int pumpSeconds = 5;
24     int controlMode = 1; // 0=MANUAL, 1=AUTO.
25     int dutyCycleMode = 0; // Fixed 30-minute cycle.
26 };
27
28 static NodeRuntimeConfig runtimeConfig;
29
30 static void loadRuntimeConfig()
31 {
32     preferences.begin("node_cfg", true);
33     runtimeConfig.soilThresholdVol = preferences.getInt("soil_th", runtimeConfig.soilThresholdVol);
34     runtimeConfig.filterMode = preferences.getInt("filter", runtimeConfig.filterMode);
35     runtimeConfig.pumpSeconds = preferences.getInt("pump_s", runtimeConfig.pumpSeconds);
36     runtimeConfig.controlMode = preferences.getInt("ctrl", runtimeConfig.controlMode);
37     preferences.end();
38     runtimeConfig.sleepMinutes = Config::DefaultSleepMinutes;
39     runtimeConfig.dutyCycleMode = 0;
40     rtcSleepMinutes = runtimeConfig.sleepMinutes;
41 }
42
43 static void saveRuntimeConfig()
44 {
45     preferences.begin("node_cfg", false);
46     preferences.putInt("soil_th", runtimeConfig.soilThresholdVol);
47     preferences.putUInt("sleep_min", Config::DefaultSleepMinutes);
48     preferences.putInt("filter", runtimeConfig.filterMode);
49     preferences.putInt("pump_s", runtimeConfig.pumpSeconds);
50     preferences.putInt("ctrl", runtimeConfig.controlMode);
51     preferences.putInt("duty", 0);
52     preferences.end();
53 }
54
55 static void setPump(bool enabled)
56 {
57     digitalWrite(Config::PumpPin, enabled == Config::PumpActiveHigh ? HIGH : LOW);
58 }
59
60 static bool initLoRa()
61 {
62     LoRa.end();
```

src/node_main.cpp

```
63     SPI.end();
64     delay(50);
65
66     pinMode(Config::LoraSs, OUTPUT);
67     digitalWrite(Config::LoraSs, HIGH);
68     pinMode(Config::LoraRst, OUTPUT);
69     digitalWrite(Config::LoraRst, LOW);
70     delay(50);
71     digitalWrite(Config::LoraRst, HIGH);
72     delay(200);
73
74     SPI.begin(Config::LoraSck, Config::LoraMiso, Config::LoraMosi, Config::LoraSs);
75     LoRa.setPins(Config::LoraSs, Config::LoraRst, Config::LoraDio0);
76     if (!LoRa.begin(Config::LoraFrequency))
77     {
78         return false;
79     }
80
81     LoRa.setSpreadingFactor(Config::LoraSpreadingFactor);
82     LoRa.setSignalBandwidth(Config::LoraSignalBandwidth);
83     LoRa.setCodingRate4(Config::LoraCodingRate);
84     LoRa.setTxPower(Config::LoraTxPowerDbm);
85     LoRa.enableCrc();
86     return true;
87 }
88
89 static DhtReading readDht22WithRetry()
90 {
91     constexpr uint8_t maxAttempts = 3;
92     for (uint8_t attempt = 1; attempt <= maxAttempts; attempt++)
93     {
94         const DhtReading reading = dht22.read();
95         if (reading.errorFlag == 0)
96         {
97             return reading;
98         }
99
100         Serial.printf("DHT read failed attempt %u/%u\n", attempt, maxAttempts);
101         delay(2000);
102     }
103
104     return DhtReading{};
105 }
106
107 static void executeCommand(const AckPacket &ack)
108 {
109     if (!ack.ok)
110     {
111         return;
112     }
113
114     switch (ack.command)
115     {
116     case CommandType::SetSleepDuration:
117         rtcSleepMinutes = Config::DefaultSleepMinutes;
118         runtimeConfig.sleepMinutes = Config::DefaultSleepMinutes;
119         saveRuntimeConfig();
120         Serial.printf("Command SET_SLEEP_DURATION ignored: fixed %lu min\n", rtcSleepMinutes);
121         break;
122     case CommandType::SetThreshold:
123         runtimeConfig.soilThresholdVol = constrain(ack.parameter, 0, 100);
124         saveRuntimeConfig();
```

src/node_main.cpp

```
125     Serial.printf("Command SET_THRESHOLD: soil_min=%d %%Vol\n", runtimeConfig.soilThresholdVol);
126     break;
127     case CommandType::SetFilterMode:
128         runtimeConfig.filterMode = constrain(ack.parameter, 0, 1);
129         saveRuntimeConfig();
130     Serial.printf("Command SET_FILTER_MODE: %s\n", runtimeConfig.filterMode == 1 ? "MEDIAN" : "A
VERAGE");
131     break;
132     case CommandType::SetPumpTime:
133         runtimeConfig.pumpSeconds = constrain(ack.parameter, 0, int(Config::MaxPumpSeconds));
134         saveRuntimeConfig();
135         Serial.printf("Command SET_PUMP_TIME: %d s\n", runtimeConfig.pumpSeconds);
136         break;
137     case CommandType::SetControlMode:
138         runtimeConfig.controlMode = constrain(ack.parameter, 0, 1);
139         saveRuntimeConfig();
140     Serial.printf("Command SET_CONTROL_MODE: %s\n", runtimeConfig.controlMode == 1 ? "AUTO" : "M
ANUAL");
141     break;
142     case CommandType::SetDutyCycle:
143         runtimeConfig.dutyCycleMode = 0;
144         saveRuntimeConfig();
145         Serial.println("Command SET_DUTY_CYCLE ignored: FIXED");
146         break;
147     case CommandType::SleepNow:
148         Serial.println("Command SLEEP_NOW");
149         break;
150     case CommandType::StartPump:
151     {
152         const int pumpSeconds = constrain(ack.parameter, 0, int(Config::MaxPumpSeconds));
153         Serial.printf("Command START_PUMP: %d s\n", pumpSeconds);
154         if (pumpSeconds > 0)
155         {
156             setPump(true);
157             delay(uint32_t(pumpSeconds) * 1000UL);
158             setPump(false);
159             Serial.println("Pump OFF");
160         }
161         break;
162     }
163     default:
164         break;
165 }
166 }
167
168 static bool waitForAck(uint32_t packetId)
169 {
170     const uint32_t start = millis();
171     while (millis() - start < Config::AckTimeoutMs)
172     {
173         const int packetSize = LoRa.parsePacket();
174         if (packetSize <= 0)
175         {
176             delay(10);
177             continue;
178         }
179
180         String line;
181         while (LoRa.available())
182         {
183             line += char(LoRa.read());
184         }
```

src/node_main.cpp

```
185
186     AckPacket ack;
187     if (!decodeAck(line, ack))
188     {
189         Serial.println("ACK invalid CRC/header");
190         Serial.print("ACK raw: ");
191         Serial.println(line);
192         continue;
193     }
194     if (ack.nodeId != Config::NodeId || ack.packetId != packetId)
195     {
196         Serial.println("ACK ignored: Node_ID/Package_ID mismatch");
197         continue;
198     }
199
200     Serial.print("ACK received: ");
201     Serial.println(line);
202     executeCommand(ack);
203     return ack.ok;
204 }
205 return false;
206 }
207
208 static bool sendTelemetry(const TelemetryPacket &telemetry)
209 {
210     const String payload = encodeTelemetry(telemetry);
211     Serial.print("LoRa TX: ");
212     Serial.println(payload);
213
214     LoRa.beginPacket();
215     LoRa.print(payload);
216     LoRa.endPacket(true);
217     delay(250);
218     LoRa.receive();
219
220     return waitForAck(telemetry.packetId);
221 }
222
223 static void enterSleepSeconds(uint32_t sleepSeconds)
224 {
225     sleepSeconds = constrain(sleepSeconds, 5UL, 90UL * 60UL);
226     rtcSleepMinutes = max(1UL, sleepSeconds / 60UL);
227     LoRa.sleep();
228     LoRa.end();
229     SPI.end();
230     digitalWrite(Config::SensorPowerPin, LOW);
231
232     if (sleepSeconds < 60)
233     {
234         Serial.printf("Sleep duration: %lu sec\n", sleepSeconds);
235     }
236     else
237     {
238         Serial.printf("Sleep duration: %lu min\n", sleepSeconds / 60UL);
239     }
240     Serial.flush();
241
242     if (Config::EnableDeepSleep)
243     {
244         esp_sleep_enable_timer_wakeup(uint64_t(sleepSeconds) * 1000000ULL);
245         esp_deep_sleep_start();
246     }
```

src/node_main.cpp

```
247 }
248
249 static void enterSleep(uint32_t sleepMinutes)
250 {
251     enterSleepSeconds(sleepMinutes * 60UL);
252 }
253
254 void setup()
255 {
256     Serial.begin(Config::SerialBaud);
257     delay(1000);
258
259     pinMode(Config::SensorPowerPin, OUTPUT);
260     digitalWrite(Config::SensorPowerPin, HIGH);
261     pinMode(Config::PumpPin, OUTPUT);
262     setPump(false);
263     delay(2000);
264
265     Serial.println("==== EE4552 NODE FIRMWARE =====");
266     Serial.printf("Wake cause: %d\n", int(esp_sleep_get_wakeup_cause()));
267     loadRuntimeConfig();
268     Serial.printf("Config: soil_threshold=%d sleep=%lu filter=%d pump=%d control=%d duty=%d\n",
269                 runtimeConfig.soilThresholdVol, runtimeConfig.sleepMinutes,
270                 runtimeConfig.filterMode, runtimeConfig.pumpSeconds,
271                 runtimeConfig.controlMode, runtimeConfig.dutyCycleMode);
272
273     dht22.begin(Config::DhtPin, Config::TempOffsetC, Config::HumidityOffsetRh);
274     soilSensor.begin(Config::SoilAdcPin, Config::SoilAdcDry, Config::SoilAdcWet,
275                     Config::SoilAdcMin, Config::SoilAdcMax);
276
277     if (!initLoRa())
278     {
279         const uint32_t failureSleepMinutes = Config::DefaultSleepMinutes;
280         Serial.printf("LoRa init failed; sleeping %lu min\n", failureSleepMinutes);
281         enterSleep(failureSleepMinutes);
282         return;
283     }
284     Serial.println("LoRa init OK");
285
286     const DhtReading dht = readDht22WithRetry();
287     const SoilReading soil = soilSensor.read();
288
289     TelemetryPacket telemetry;
290     telemetry.nodeId = Config::NodeId;
291     telemetry.packetId = ++rtcPacketId;
292     telemetry.temperatureC = dht.temperatureC;
293     telemetry.humidityRh = dht.humidityRh;
294     telemetry.soilMoistureVol = soil.moistureVol;
295     telemetry.adcFiltered = soil.adcFiltered;
296     telemetry.errorFlag = dht.errorFlag || soil.errorFlag;
297     telemetry.soilStatus = SoilMoistureSensor::statusText(soil.soilStatus);
298     telemetry.configSleepMinutes = runtimeConfig.sleepMinutes;
299     telemetry.configSoilThresholdVol = runtimeConfig.soilThresholdVol;
300     telemetry.configFilterMode = runtimeConfig.filterMode;
301     telemetry.configPumpSeconds = runtimeConfig.pumpSeconds;
302     telemetry.configControlMode = runtimeConfig.controlMode;
303     telemetry.configDutyCycleMode = runtimeConfig.dutyCycleMode;
304
305     Serial.printf("DHT: T=%.1f C H=%.1f %%RH ERR=%u\n", telemetry.temperatureC,
306                 telemetry.humidityRh, dht.errorFlag);
307     Serial.printf("SOIL: ADC=%d H=%.1f %%Vol status=%s ERR=%u\n", telemetry.adcFiltered,
308                 telemetry.soilMoistureVol, telemetry.soilStatus.c_str(), soil.errorFlag);
```

src/node_main.cpp

```
309
310     const bool delivered = sendTelemetry(telemetry);
311     if (!delivered)
312     {
313         Serial.println("No ACK; TX failed");
314     }
315
316     enterSleep(Config::DefaultSleepMinutes);
317 }
318
319 void loop()
320 {
321     if (!Config::EnableDeepSleep)
322     {
323         delay(uint32_t(rtcSleepMinutes) * 60UL * 1000UL);
324         ESP.restart();
325     }
326 }
```

src/packet_protocol.cpp

```
1  #include "packet_protocol.h"
2
3  static String withoutCrc(const String &line)
4  {
5      const int crcIndex = line.lastIndexOf(",CRC=");
6      if (crcIndex < 0)
7      {
8          return line;
9      }
10     return line.substring(0, crcIndex);
11 }
12
13 uint16_t crc16Ccitt(const uint8_t *data, size_t length)
14 {
15     uint16_t crc = 0xFFFF;
16     for (size_t i = 0; i < length; i++)
17     {
18         crc ^= uint16_t(data[i]) << 8;
19         for (uint8_t bit = 0; bit < 8; bit++)
20         {
21             crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : crc << 1;
22         }
23     }
24     return crc;
25 }
26
27 uint16_t crc16Ccitt(const String &text)
28 {
29     return crc16Ccitt(reinterpret_cast<const uint8_t *>(text.c_str()), text.length());
30 }
31
32 String commandToText(CommandType command)
33 {
34     switch (command)
35     {
36     case CommandType::SetSleepDuration:
37         return "SET_SLEEP_DURATION";
38     case CommandType::SetThreshold:
39         return "SET_THRESHOLD";
40     case CommandType::SleepNow:
41         return "SLEEP_NOW";
42     case CommandType::StartPump:
43         return "START_PUMP";
44     case CommandType::SetFilterMode:
45         return "SET_FILTER_MODE";
46     case CommandType::SetPumpTime:
47         return "SET_PUMP_TIME";
48     case CommandType::SetControlMode:
49         return "SET_CONTROL_MODE";
50     case CommandType::SetDutyCycle:
51         return "SET_DUTY_CYCLE";
52     default:
53         return "NONE";
54     }
55 }
56
57 CommandType commandFromText(const String &text)
58 {
59     if (text == "SET_SLEEP_DURATION")
60         return CommandType::SetSleepDuration;
61     if (text == "SET_THRESHOLD")
62         return CommandType::SetThreshold;
```


src/packet_protocol.cpp

```
63     if (text == "SLEEP_NOW")
64         return CommandType::SleepNow;
65     if (text == "START_PUMP")
66         return CommandType::StartPump;
67     if (text == "SET_FILTER_MODE")
68         return CommandType::SetFilterMode;
69     if (text == "SET_PUMP_TIME")
70         return CommandType::SetPumpTime;
71     if (text == "SET_CONTROL_MODE")
72         return CommandType::SetControlMode;
73     if (text == "SET_DUTY_CYCLE")
74         return CommandType::SetDutyCycle;
75     return CommandType::None;
76 }
77
78 String encodeTelemetry(const TelemetryPacket &packet)
79 {
80     String base = "TYPE=DATA";
81     base += ",NODE=" + String(packet.nodeId);
82     base += ",PID=" + String(packet.packetId);
83     base += ",T=" + String(packet.temperatureC, 1);
84     base += ",HA=" + String(packet.humidityRh, 1);
85     base += ",HS=" + String(packet.soilMoistureVol, 0);
86     base += ",ADC=" + String(packet.adcFiltered);
87     base += ",SOIL=" + packet.soilStatus;
88     base += ",ERR=" + String(packet.errorFlag);
89     base += ",CSLEEP=" + String(packet.configSleepMinutes);
90     base += ",CTH=" + String(packet.configSoilThresholdVol);
91     base += ",CFILTER=" + String(packet.configFilterMode);
92     base += ",CPUMP=" + String(packet.configPumpSeconds);
93     base += ",CMODE=" + String(packet.configControlMode);
94     base += ",CDUTY=" + String(packet.configDutyCycleMode);
95     base.toUpperCase();
96     base += ",CRC=" + String(crc16Ccitt(base), HEX);
97     base.toUpperCase();
98     return base;
99 }
100
101 String encodeAck(const AckPacket &packet)
102 {
103     String base = "TYPE=ACK";
104     base += ",NODE=" + String(packet.nodeId);
105     base += ",PID=" + String(packet.packetId);
106     base += ",OK=" + String(packet.ok ? 1 : 0);
107     base += ",CMD=" + commandToText(packet.command);
108     base += ",PARAM=" + String(packet.parameter);
109     base += ",STATUS=" + packet.status;
110     base.toUpperCase();
111     base += ",CRC=" + String(crc16Ccitt(base), HEX);
112     base.toUpperCase();
113     return base;
114 }
115
116 bool decodeTelemetry(const String &line, TelemetryPacket &packet)
117 {
118     if (getField(line, "TYPE") != "DATA" || !hasValidCrc(line))
119     {
120         return false;
121     }
122     packet.nodeId = uint8_t(getField(line, "NODE").toInt());
123     packet.packetId = uint32_t(getField(line, "PID").toInt());
124     packet.temperatureC = getField(line, "T").toFloat();
```

src/packet_protocol.cpp

```
125     packet.humidityRh = getField(line, "HA").toFloat();
126     packet.soilMoistureVol = getField(line, "HS").toFloat();
127     packet.adcFiltered = getField(line, "ADC").toInt();
128     packet.soilStatus = getField(line, "SOIL");
129     packet.errorFlag = uint8_t(getField(line, "ERR").toInt());
130     packet.configSleepMinutes = uint32_t(getField(line, "CSLEEP").toInt());
131     packet.configSoilThresholdVol = getField(line, "CTH").toInt();
132     packet.configFilterMode = getField(line, "CFILTER").toInt();
133     packet.configPumpSeconds = getField(line, "CPUMP").toInt();
134     packet.configControlMode = getField(line, "CMODE").toInt();
135     packet.configDutyCycleMode = getField(line, "CDUTY").toInt();
136     return packet.nodeId > 0 && packet.packetId > 0;
137 }
138
139 bool decodeAck(const String &line, AckPacket &packet)
140 {
141     if (getField(line, "TYPE") != "ACK" || !hasValidCrc(line))
142     {
143         return false;
144     }
145     packet.nodeId = uint8_t(getField(line, "NODE").toInt());
146     packet.packetId = uint32_t(getField(line, "PID").toInt());
147     packet.ok = getField(line, "OK").toInt() == 1;
148     packet.command = commandFromText(getField(line, "CMD"));
149     packet.parameter = getField(line, "PARAM").toInt();
150     packet.status = getField(line, "STATUS");
151     return packet.nodeId > 0 && packet.packetId > 0;
152 }
153
154 bool hasValidCrc(const String &line)
155 {
156     const String expected = getField(line, "CRC");
157     if (expected.length() == 0)
158     {
159         return false;
160     }
161     const uint16_t actual = crc16Citt(withoutCrc(line));
162     return uint16_t(strtoul(expected.c_str(), nullptr, 16)) == actual;
163 }
164
165 String getField(const String &line, const String &key)
166 {
167     const String token = key + "=";
168     int start = -1;
169     if (line.startsWith(token))
170     {
171         start = 0;
172     }
173     else
174     {
175         start = line.indexOf(", " + token);
176         if (start >= 0)
177         {
178             start += 1;
179         }
180     }
181
182     if (start < 0)
183     {
184         return "";
185     }
186     start += token.length();
```

src/packet_protocol.cpp

```
187     int end = line.indexOf(',', start);
188     if (end < 0)
189     {
190         end = line.length();
191     }
192     return line.substring(start, end);
193 }
```

src/packet_protocol.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  enum class CommandType : uint8_t
6  {
7      None = 0,
8      SetSleepDuration = 1,
9      SetThreshold = 2,
10     SleepNow = 4,
11     StartPump = 5,
12     SetFilterMode = 7,
13     SetPumpTime = 8,
14     SetControlMode = 9,
15     SetDutyCycle = 10
16 };
17
18 enum class PacketKind : uint8_t
19 {
20     Telemetry = 1,
21     Ack = 2,
22     Command = 3
23 };
24
25 struct TelemetryPacket
26 {
27     uint8_t nodeId = 0;
28     uint32_t packetId = 0;
29     float temperatureC = NAN;
30     float humidityRh = NAN;
31     float soilMoistureVol = NAN;
32     int adcFiltered = 0;
33     uint8_t errorFlag = 0;
34     String soilStatus;
35     uint32_t configSleepMinutes = 0;
36     int configSoilThresholdVol = 0;
37     int configFilterMode = 0;
38     int configPumpSeconds = 0;
39     int configControlMode = 0;
40     int configDutyCycleMode = 0;
41 };
42
43 struct AckPacket
44 {
45     uint8_t nodeId = 0;
46     uint32_t packetId = 0;
47     bool ok = false;
48     CommandType command = CommandType::None;
49     int parameter = 0;
50     String status;
51 };
52
53 uint16_t crc16Ccitt(const uint8_t *data, size_t length);
54 uint16_t crc16Ccitt(const String &text);
55
56 String commandToText(CommandType command);
57 CommandType commandFromText(const String &text);
58
59 String encodeTelemetry(const TelemetryPacket &packet);
60 String encodeAck(const AckPacket &packet);
61 bool decodeTelemetry(const String &line, TelemetryPacket &packet);
62 bool decodeAck(const String &line, AckPacket &packet);
```

src/packet_protocol.h

```
63 bool hasValidCrc(const String &line);  
64 String getField(const String &line, const String &key);
```

src/project_config.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  namespace Config
6  {
7      constexpr uint32_t SerialBaud = 115200;
8      constexpr char GatewayFirmwareVersion[] = "GW_1";
9
10     constexpr uint8_t NodeId = 1;
11     constexpr uint8_t GatewayId = 1;
12
13     constexpr uint8_t DhtPin = 27;
14     constexpr uint8_t SoilAdcPin = 34;
15     constexpr uint8_t SensorPowerPin = 32;
16     constexpr uint8_t PumpPin = 25;
17     constexpr bool PumpActiveHigh = true;
18     constexpr uint8_t MaxPumpSeconds = 15;
19     constexpr bool EnablePumpHardware = false;
20
21     constexpr float TempOffsetC = 0.0f;
22     constexpr float HumidityOffsetRh = 0.0f;
23     constexpr int SoilAdcDry = 3400;
24     constexpr int SoilAdcWet = 1200;
25     constexpr int SoilAdcMin = 100;
26     constexpr int SoilAdcMax = 4090;
27
28     constexpr uint8_t LoraSs = 5;
29     constexpr uint8_t LoraRst = 14;
30     constexpr uint8_t LoraDio0 = 26;
31     constexpr uint8_t LoraSck = 18;
32     constexpr uint8_t LoraMiso = 19;
33     constexpr uint8_t LoraMosi = 23;
34     constexpr long LoraFrequency = 433E6;
35     constexpr uint8_t LoraSpreadingFactor = 7;
36     constexpr long LoraSignalBandwidth = 125E3;
37     constexpr uint8_t LoraCodingRate = 5;
38     constexpr uint8_t LoraTxPowerDbm = 17;
39
40     constexpr uint16_t AckTimeoutMs = 2500;
41     constexpr uint16_t CommandDedupWindow = 32;
42
43     constexpr bool EnableDeepSleep = true;
44     constexpr uint32_t DefaultSleepMinutes = 30;
45
46     constexpr char WifiSsid[] = "Khoa";
47     constexpr char WifiPassword[] = "12112004";
48     constexpr char ThingsBoardHost[] = "eu.thingsboard.cloud";
49     constexpr char ThingsBoardToken[] = "QYtNEEPckyFjPteWRQ9o";
50     constexpr uint16_t ThingsBoardHttpPort = 80;
51     constexpr bool EnableCloudUpload = true;
52
53     constexpr bool EnableGatewayHttpDebug = false;
54     constexpr bool EnableGatewayHeartbeatLog = false;
55 }
```

src/soil_moisture.cpp

```
1  #include "soil_moisture.h"
2
3  #include <math.h>
4
5  void SoilMoistureSensor::begin(uint8_t adcPin, int adcDry, int adcWet, int adcMin, int adcMax)
6  {
7      _adcPin = adcPin;
8      _adcDry = adcDry;
9      _adcWet = adcWet;
10     _adcMin = adcMin;
11     _adcMax = adcMax;
12     analogReadResolution(12);
13     analogSetPinAttenuation(_adcPin, ADC_11db);
14 }
15
16 SoilReading SoilMoistureSensor::read()
17 {
18     SoilReading reading;
19     int samples[kSamples - kDiscardFirst] = {};
20
21     for (size_t i = 0; i < kSamples; i++)
22     {
23         const int adc = analogRead(_adcPin);
24         reading.adcRaw = adc;
25
26         if (adc <= _adcMin || adc >= _adcMax)
27         {
28             _adcErrorCount++;
29             if (_adcErrorCount >= kMaxAdcErrors)
30             {
31                 reading.soilStatus = SoilStatus::Error;
32                 reading.sensorStatus = SensorStatus::Error;
33                 reading.errorFlag = 1;
34                 return reading;
35             }
36             delay(350);
37             i--;
38             continue;
39         }
40
41         if (i >= kDiscardFirst)
42         {
43             samples[i - kDiscardFirst] = adc;
44         }
45         delay(20);
46     }
47
48     _adcErrorCount = 0;
49     reading.adcFiltered = median(samples, kSamples - kDiscardFirst);
50     reading.moistureVol = clampMoisture(
51         (_adcDry - reading.adcFiltered) * 60.0f / float(_adcDry - _adcWet));
52     reading.soilStatus = classify(reading.moistureVol);
53     reading.sensorStatus = SensorStatus::Ok;
54     reading.errorFlag = 0;
55     return reading;
56 }
57
58 const char *SoilMoistureSensor::statusText(SoilStatus status)
59 {
60     switch (status)
61     {
62     case SoilStatus::UrgentWatering:
```

src/soil_moisture.cpp

```
63     return "URGENT_WATERING";
64 case SoilStatus::NeedWatering:
65     return "NEED_WATERING";
66 case SoilStatus::LightDry:
67     return "LIGHT_DRY";
68 case SoilStatus::Normal:
69     return "NORMAL";
70 case SoilStatus::OverMoisture:
71     return "OVER_MOISTURE";
72 default:
73     return "ERROR";
74 }
75 }
76
77 SoilStatus SoilMoistureSensor::classify(float moistureVol)
78 {
79     if (isnan(moistureVol))
80     {
81         return SoilStatus::Error;
82     }
83     if (moistureVol > 42.0f)
84     {
85         return SoilStatus::OverMoisture;
86     }
87     if (moistureVol > 33.0f)
88     {
89         return SoilStatus::Normal;
90     }
91     if (moistureVol > 24.0f)
92     {
93         return SoilStatus::LightDry;
94     }
95     if (moistureVol > 15.0f)
96     {
97         return SoilStatus::NeedWatering;
98     }
99     return SoilStatus::UrgentWatering;
100 }
101
102 int SoilMoistureSensor::median(int *values, size_t length)
103 {
104     for (size_t i = 1; i < length; i++)
105     {
106         const int key = values[i];
107         int j = int(i) - 1;
108         while (j >= 0 && values[j] > key)
109         {
110             values[j + 1] = values[j];
111             j--;
112         }
113         values[j + 1] = key;
114     }
115     return values[length / 2];
116 }
117
118 float SoilMoistureSensor::clampMoisture(float moistureVol)
119 {
120     if (moistureVol < 0.0f)
121     {
122         return 0.0f;
123     }
124     if (moistureVol > 60.0f)
```


src/soil_moisture.cpp

```
125     {  
126         return 60.0f;  
127     }  
128     return roundf(moistureVol * 10.0f) / 10.0f;  
129 }
```

src/soil_moisture.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  #include "dht22_sensor.h"
6
7  enum class SoilStatus : uint8_t
8  {
9      UrgentWatering = 0,
10     NeedWatering = 1,
11     LightDry = 2,
12     Normal = 3,
13     OverMoisture = 4,
14     Error = 5
15 };
16
17 struct SoilReading
18 {
19     int adcRaw = 0;
20     int adcFiltered = 0;
21     float moistureVol = NAN;
22     SoilStatus soilStatus = SoilStatus::Error;
23     SensorStatus sensorStatus = SensorStatus::Error;
24     uint8_t errorFlag = 1;
25 };
26
27 class SoilMoistureSensor
28 {
29 public:
30     void begin(uint8_t adcPin, int adcDry, int adcWet, int adcMin, int adcMax);
31     SoilReading read();
32
33     static const char *statusText(SoilStatus status);
34     static SoilStatus classify(float moistureVol);
35
36 private:
37     static constexpr size_t kSamples = 11;
38     static constexpr size_t kDiscardFirst = 3;
39     static constexpr uint8_t kMaxAdcErrors = 3;
40
41     uint8_t _adcPin = 0;
42     int _adcDry = 3400;
43     int _adcWet = 1200;
44     int _adcMin = 100;
45     int _adcMax = 4090;
46     uint8_t _adcErrorCount = 0;
47
48     static int median(int *values, size_t length);
49     static float clampMoisture(float moistureVol);
50 };
```